


Universidad Latina de Costa Rica.

# Proyecto Tetris

Jafet Campos Agüero, Meydelline Guerrero López.

Programación V.

Osvaldo Fabián Granados

Octubre 28, 2026.

Introducción ..... 5

Objetivo del manual técnico ..... 5

Descripción general del sistema ..... 5

Solitario ..... 6

Compartido y Competitivo ..... 6

Funcionamiento general ..... 7

Tecnologías utilizadas ..... 7

Angular e Ionic ..... 7

ASP.NET Core y C# ..... 7

SQL Server ..... 8

SignalR, Firebase y Cloudflare ..... 8

Arquitectura del sistema ..... 8

Frontend ..... 9

API ..... 9

Lógica de negocios ..... 9

Acceso a datos ..... 10

Base de datos ..... 10

Flujo general ..... 10

Estructura del proyecto ..... 10

- Frontend Ionic/Angular ..... 11
- ProyectoTetris_PrograV (API) ..... 11
- LogicaNegocio ..... 11
- AccesoDatos ..... 11

Dominio ..... 12

Utilitarios ..... 12

Tetris.Presentacion ..... 12

Tetris.Pruebas ..... 12

Base de datos ..... 13

Tablas principales ..... 13

Usuarios: ..... 13

CredencialesDispositivo: ..... 13

ConfiguracionJugador: ..... 13

EstadisticasJugador: ..... 14

Partidas: ..... 14

ParticipantesPartida: ..... 14

Resultados: ..... 14

TiposPieza: ..... 14

Relaciones entre las tablas ..... 14

Backend ..... 15

Entidades ..... 16

Entidades tipadas ..... 16

Interfaces ..... 17

Lógica de negocio ..... 17

Controladores/API ..... 18

Frontend ..... 18

Modelos ..... 18

Servicio ..... 19

Componentes ..... 19

Formularios ..... 20

Rutas ..... 20

Autenticación y sesiones ..... 20

Cuentas de jugador ..... 21

Google y recuperación ..... 21

Guards y permisos ..... 21

Módulos del sistema ..... 22

Solitario ..... 22

Compartido ..... 22

Competitivo ..... 23

Piezas especiales ..... 23

Pausas y reconexión ..... 23

Resultados y configuración ..... 24

Publicación y ejecución ..... 24

Conclusión ..... 25

## Introducción

Este manual técnico tiene como objetivo explicar de una manera clara cómo está desarrollado y cómo funciona Proyecto Tetris. En este documento se describen las principales partes que conforman el juego, como el frontend, el backend, la base de datos y la comunicación que existe entre estos componentes.

Proyecto Tetris es un videojuego desarrollado como proyecto universitario. Permite jugar de forma individual o participar con otras personas mediante salas. Incluye registro de jugadores, puntuaciones, niveles, piezas especiales en la modalidad correspondiente y opciones para continuar una partida si se interrumpe la conexión.

El manual también busca servir como una guía para comprender la organización del código y facilitar futuras correcciones o mejoras. La información se presenta por secciones, utilizando nombres y ejemplos que corresponden a los archivos reales del proyecto.

## Objetivo del manual técnico

El objetivo de este manual es documentar la estructura y el funcionamiento de Proyecto Tetris, explicando las tecnologías utilizadas, la organización del proyecto, sus componentes principales y la forma en que se relacionan.

De esta manera, una persona que necesite revisar, modificar o darle mantenimiento al juego podrá comprender con mayor facilidad cómo está construido y qué responsabilidad tiene cada parte del sistema.

## Descripción general del sistema

Proyecto Tetris es un juego de bloques que van cayendo dentro de un tablero. El jugador debe mover y acomodar las piezas para formar filas completas. Cuando una fila se completa,

desaparece y se obtiene puntuación. El proyecto incorpora distintas modalidades para jugar solo o con otras personas.

El sistema maneja jugadores identificados por un nombre de usuario. Desde la aplicación pueden acceder al menú principal, consultar su perfil, ajustar algunas opciones y entrar a las partidas disponibles.

### Solitario

En este modo participa únicamente un jugador y utiliza un tablero individual. Las piezas van cayendo y la partida continúa mientras haya espacio para colocar nuevas piezas. El nivel aumenta según la cantidad de filas eliminadas, y el jugador puede intentar superar su mejor puntuación. Como no permite invitados, Solitario no utiliza un código para compartir la partida.

### Compartido y Competitivo

Compartido permite jugar de dos a cuatro personas sobre un mismo tablero. Cada participante juega cuando le corresponde su turno y las filas normales eliminadas permiten aumentar el nivel del tablero común. El creador puede configurar piezas especiales y otras opciones antes de iniciar.

Competitivo también admite de dos a cuatro jugadores, pero cada persona tiene su propio tablero y juega al mismo tiempo. En este modo no se utilizan piezas especiales. El ganador se determina por la puntuación final; si hay empate, se reconoce a los jugadores con el mismo primer lugar.

## Funcionamiento general

De forma general, el frontend muestra la partida y recibe las acciones del jugador. El backend comprueba que los movimientos sean válidos, aplica las reglas y comunica los resultados. SQL Server almacena las cuentas, las estadísticas y los resultados históricos; los tableros de las partidas activas se mantienen en la memoria del servidor.

## Tecnologías utilizadas

Para desarrollar Proyecto Tetris se utilizaron diferentes herramientas que permiten trabajar con la parte visual, las reglas del juego, la comunicación entre jugadores y el almacenamiento de información.

## Angular e Ionic

Angular se utilizó para desarrollar las páginas, los componentes y los servicios de la aplicación. Ionic permite adaptar esa interfaz a pantallas de celulares. Además, Capacitor permite llevar el mismo frontend a Android y generar una aplicación instalable.

## ASP.NET Core y C#

El backend fue desarrollado con C# y ASP.NET Core sobre .NET 10. Esta parte recibe las solicitudes, valida los datos de los jugadores, controla las salas y ejecuta las reglas de cada modalidad. También se comunica con SQL Server cuando necesita consultar o guardar información.

## SQL Server

SQL Server se utiliza para almacenar los usuarios, credenciales de dispositivo, preferencias, partidas terminadas, resultados y estadísticas. Entity Framework Core ayuda a relacionar las clases de C# con las tablas de la base de datos.

## SignalR, Firebase y Cloudflare

SignalR permite enviar los cambios de una partida a los jugadores sin que tengan que actualizar la pantalla manualmente. Firebase Hosting publica la aplicación web y Cloudflare Tunnel facilita que los celulares se conecten por Internet con la API que se ejecuta en la computadora anfitriona.

## Arquitectura del sistema

Proyecto Tetris está organizado por capas para separar las responsabilidades del programa. Así, las pantallas no tienen que encargarse directamente de las reglas del juego o de guardar información en SQL Server. Esta organización facilita localizar errores y modificar una parte sin tener que cambiar todas las demás.

El funcionamiento inicia cuando el jugador realiza una acción desde la aplicación. El frontend envía la solicitud al backend y este revisa si la operación es válida. Cuando se trata de una partida, el servidor actualiza el estado del juego y lo envía a los participantes. Cuando se necesita guardar información, utiliza la capa de acceso a datos.

## Frontend

El frontend se desarrolló con Angular e Ionic y corresponde a la parte visual con la que interactúan los jugadores. Contiene el menú principal, registro e ingreso, lista de salas, pantallas de partida, perfil, ajustes y los controles necesarios para mover las piezas.

También tiene servicios para llamar a la API y recibir información mediante SignalR. Los componentes muestran el tablero, la próxima pieza, los puntajes y los mensajes que corresponden al estado actual.

## API

La API fue desarrollada con ASP.NET Core y funciona como el punto de comunicación entre la aplicación y el backend. Los controladores reciben las solicitudes relacionadas con usuarios, salas, configuraciones y resultados, y utilizan los servicios que realizan las operaciones.

Algunas acciones requieren que el jugador tenga una sesión válida. Además, el servidor comprueba que solo las personas autorizadas puedan realizar movimientos, modificar la configuración de una sala o iniciar una partida.

## Lógica de negocios

La capa de lógica de negocios contiene las reglas y procesos necesarios para el funcionamiento del juego. Aquí se revisan los movimientos, los turnos, las puntuaciones, la eliminación de filas, los efectos de las piezas especiales y las condiciones que determinan cuándo termina una ronda.

Esta capa permite mantener las reglas fuera de los controladores y evita que el frontend tenga que decidir los resultados de una partida.

## Acceso a datos

La capa de acceso a datos se encarga de consultar y guardar información mediante Entity Framework Core. El contexto TetrisDBContext representa las tablas y sus relaciones, mientras que los repositorios y la unidad de trabajo ayudan a realizar las operaciones necesarias.

Por ejemplo, cuando finaliza una ronda, los resultados y las estadísticas se guardan utilizando estos servicios. Los movimientos de cada pieza no requieren escribir todas las posiciones en SQL Server.

## Base de datos

La base de datos de Proyecto Tetris utiliza SQL Server y mantiene la información que debe conservarse aunque se cierre la aplicación. Entre sus datos principales se encuentran usuarios, configuraciones, estadísticas, partidas, participantes y resultados.

## Flujo general

El proceso puede resumirse de esta forma: el jugador utiliza Angular e Ionic; la aplicación se comunica con los controladores de la API; la lógica de negocio procesa la operación; y, cuando es necesario, el acceso a datos consulta o actualiza SQL Server.

En las partidas multijugador, SignalR devuelve las actualizaciones a quienes pertenecen a la sala. De esta manera, todos pueden ver el estado que el servidor considera correcto.

## Estructura del proyecto

Proyecto Tetris se encuentra dividido en un frontend y una solución de backend. Dentro de ellos existen carpetas y proyectos con responsabilidades diferentes. Esto mantiene el código organizado y facilita ubicar dónde se encuentra cada función.

Los componentes principales de esta organización son los siguientes:

**Frontend Ionic/Angular**

Se encuentra en la carpeta frontend/tetris-app. Contiene las páginas, componentes, modelos, servicios, rutas, imágenes y sonidos que utiliza la aplicación. También incluye la configuración para compilar la versión web y el proyecto Android de Capacitor.

**ProyectoTetris_PrograV (API)**

Este proyecto contiene la aplicación ASP.NET Core, sus controladores, servicios, configuración de inicio, seguridad y el hub de SignalR. Su función es recibir las solicitudes del frontend y comunicarse con las demás partes del backend.

**LogicaNegocio**

Contiene los servicios y procesos que aplican las reglas del juego, como SalaLJ, MotorJuego, los controles de pausa y efectos especiales, y los procesos que guardan los resultados. Se encarga de que las acciones de los jugadores se realicen de acuerdo con las reglas.

**AccesoDatos**

En este proyecto se encuentra TetrisDBContext, además de los repositorios y la unidad de trabajo. Estas clases permiten consultar, insertar, modificar y guardar información en SQL Server de una manera organizada.

**Dominio**

Contiene las entidades, entidades tipadas, interfaces y clases que representan los datos principales del sistema. También incluye las estructuras del tablero y de las piezas, que son utilizadas por la lógica de negocio.

Por ejemplo, TableroJuego representa la información necesaria para manejar un tablero, mientras que los contratos de sala permiten enviar al frontend la información que el jugador debe visualizar.

**Utilitarios**

Este proyecto contiene clases de apoyo que utilizan varias partes del backend. Entre ellas se encuentra Respuesta, que permite indicar si una operación fue correcta, devolver los datos obtenidos o enviar un mensaje cuando existe algún problema.

**Tetris.Presentacion**

Es el proyecto que facilita iniciar el sistema para las demostraciones. Revisa los recursos necesarios, comprueba SQL Server, inicia la API y el túnel público, y muestra un panel local para conocer el estado de la conexión.

Cuando se necesita actualizar la página web, solicita escribir PUBLICAR antes de ejecutar la publicación de Firebase. De esta forma no se actualiza el sitio sin una confirmación.

**Tetris.Pruebas**

Incluye comprobaciones que ayudan a revisar las reglas del juego, el manejo de las salas, la autenticación y la comunicación. Estas pruebas permiten detectar problemas antes de preparar una nueva versión del programa.

La solución también contiene un cliente HTTP en C# y varias herramientas de apoyo. El frontend que utilizan los jugadores es la aplicación Ionic/Angular.

**Base de datos**

Proyecto Tetris utiliza SQL Server para almacenar y administrar los datos que deben conservarse. La base se identifica como TetrisDB y sus registros se relacionan principalmente con las cuentas de los jugadores y los resultados de las partidas.

Las salas y los tableros activos se manejan en memoria para responder a los movimientos con rapidez. Al terminar una ronda, el sistema registra la información histórica correspondiente en la base de datos.

**Tablas principales**

Usuarios:

almacena la información principal de las cuentas, como el identificador, el nombre de usuario, la fecha de registro y los datos necesarios para reconocer una sesión o una cuenta vinculada con Google.

**CredencialesDispositivo:**

guarda las huellas de las credenciales utilizadas para ingresar desde un dispositivo, junto con su vigencia y posible revocación. No almacena la credencial original en texto plano.

**ConfiguracionJugador:**

conserva opciones asociadas con un usuario, como preferencias generales del juego.

**EstadisticasJugador:**

guarda las partidas jugadas, ganadas y perdidas, la mejor puntuación, las líneas eliminadas y otros valores acumulados.

**Partidas:**

almacena información histórica de las rondas, su creador, modalidad, estado, fechas y posible ganador.

**ParticipantesPartida:**

relaciona a los jugadores con las partidas en las que participaron y conserva algunos datos de su participación.

**Resultados:**

registra la puntuación final, líneas, nivel, posición y condición de ganador de cada participante.

**TiposPieza:**

mantiene la información del catálogo de piezas y efectos especiales que puede utilizar el sistema.

## Relaciones entre las tablas

Las tablas de Proyecto Tetris se encuentran relacionadas para que la información permanezca organizada y los registros correspondan al jugador o a la partida correcta.

Por ejemplo, un registro de Usuarios puede tener una configuración y unas estadísticas asociadas. Un usuario también puede contar con credenciales de dispositivo para ingresar a su cuenta y recuperar su sesión.

Por otra parte, una partida se relaciona con los participantes que jugaron en ella. Cada participante puede tener un resultado asociado, donde se conserva su puntuación y la información del final de la ronda.

Las relaciones entre Partidas, ParticipantesPartida y Resultados permiten consultar el historial de una partida sin repetir todos los datos del usuario en cada registro. También hacen posible mostrar estadísticas en el perfil del jugador.

En el caso de un empate, más de un resultado puede indicar que el jugador obtuvo el primer lugar. Por eso no se debe asumir que siempre existe un solo ganador en los resultados guardados.

La estructura incluye otras tablas históricas relacionadas con turnos, tableros, piezas y eventos. Sin embargo, que aparezcan en el esquema no significa que el motor actual guarde cada movimiento en esas tablas.

Las relaciones y restricciones se encuentran descritas en las entidades y en TetrisDBContext. Para comprobar el estado exacto de una base instalada, se debe revisar también SQL Server.

**Backend**

El backend de Proyecto Tetris fue desarrollado con ASP.NET Core y C#. Esta parte se encarga de procesar las solicitudes recibidas desde el frontend, aplicar las reglas necesarias y trabajar con la base de datos cuando corresponde.

Para mantener el código organizado se utilizan entidades, entidades tipadas, interfaces, lógica de negocio, controladores y servicios de comunicación en tiempo real.

## Entidades

Las entidades representan la información que se guarda en la base de datos y que puede utilizarse dentro del programa. Por ejemplo, Usuario contiene propiedades relacionadas con la cuenta del jugador y Resultado representa los datos de una partida terminada.

Una entidad puede contener propiedades como estas:

```
public int UsuarioId { get; set; }
public string NombreUsuario { get; set; }
public bool Activo { get; set; }
```

Estas propiedades permiten que el programa trabaje con los datos almacenados en SQL Server a través de las clases de C#.

## Entidades tipadas

Las entidades tipadas se utilizan para transportar la información que necesita una operación o una pantalla, sin enviar necesariamente toda la entidad que se guarda en la base de datos.

Por ejemplo, el resultado de una operación con salas puede devolverse como Respuesta<TSala>. Esta respuesta indica si la acción salió bien y contiene la información de la sala que el frontend necesita mostrar.

## Interfaces

Las interfaces definen las operaciones que debe ofrecer cada parte del backend y ayudan a organizar la comunicación entre clases. Se utilizan para establecer qué puede hacer un servicio sin obligar al controlador a conocer todos los detalles de su implementación.

Por ejemplo, las interfaces de lógica de juego y acceso a datos indican las funciones disponibles para trabajar con salas, usuarios, resultados y repositorios.

## Lógica de negocio

La lógica de negocio se encarga de revisar las condiciones necesarias antes de realizar una acción. El servicio SalaLJ administra las salas y sus participantes, y MotorJuego trabaja con movimientos, colisiones, filas y puntuación.

En la creación de una sala, por ejemplo, el código valida el modo y la capacidad. Las partidas individuales permiten un jugador, mientras que las salas multijugador admiten de dos a cuatro. El código de invitación se reserva únicamente para las modalidades multijugador.

```
// Solitario no recibe código de invitación.
var esIndividual = datos.Modo == ModoJuego.Individual;
if (!esIndividual)
{
var codigo = codigosSala.Reservar(sala.Id);
if (codigo == null)
{
return Respuesta<TSala>.Fail("No se pudo generar el código de la sala. Intente de nuevo.");
}
```

```
sala.CodigoIngreso = codigo;
}
```

## Controladores/API

Los controladores reciben solicitudes del frontend y utilizan los servicios correspondientes para completarlas. Por ejemplo, la API dispone de operaciones para registrar usuarios, crear salas, unirse por código, indicar que un jugador está listo y consultar los resultados.

Las acciones protegidas utilizan la identidad de la sesión del jugador. El servidor vuelve a revisar las condiciones de una jugada, ya que no debe confiar solamente en las validaciones que realiza la aplicación visual.

## Frontend

El frontend de Proyecto Tetris fue desarrollado con Angular e Ionic y corresponde a la parte visual con la que interactúan los jugadores. Se encuentra organizado mediante modelos, servicios, páginas, componentes y rutas.

Estas partes trabajan juntas para mostrar el menú, administrar el ingreso, crear o buscar salas y visualizar el tablero y sus cambios durante una partida.

## Modelos

Los modelos representan los datos que Angular envía y recibe desde la API. Por ejemplo, los modelos de tablero contienen la información que se necesita para dibujar las celdas, la pieza activa y otros datos de la partida.

También se utilizan tipos e interfaces para representar jugadores, salas, movimientos, configuraciones y resultados. Esto permite trabajar con la información de una forma más ordenada.

## Servicio

Los servicios permiten que las páginas se comuniquen con el backend y compartan información. ApiServicio se encarga de enviar solicitudes HTTP; AutenticacionServicio trabaja con la sesión del jugador y SalasServicio maneja las acciones relacionadas con las salas.

Por ejemplo, el método enviar de ApiServicio prepara la solicitud, agrega la autorización cuando corresponde y revisa la respuesta que devuelve el servidor. Así, las páginas no necesitan repetir esa lógica.

El frontend también utiliza servicios para audio, controles y conexión con SignalR. Cuando llega una actualización de la sala, la pantalla muestra la información más reciente.

## Componentes

Los componentes representan las partes de la pantalla que se reutilizan dentro del juego. Por ejemplo, TableroComponent se encarga de dibujar el tablero; SiguientePiezaComponent muestra la próxima pieza; y ControlesComponent permite realizar movimientos desde la pantalla del celular.

También existen componentes para mostrar el código de una sala, configurar opciones y avisar cuando el dispositivo se encuentra en una orientación que no es adecuada para jugar.

## Formularios

Los formularios permiten ingresar información y modificar algunas opciones del sistema. Se utilizan para registrar el nickname, recuperar una cuenta, crear salas y establecer configuraciones antes de iniciar una partida.

Antes de enviar información se comprueba que cumpla ciertas condiciones, pero el backend también realiza sus propias validaciones. Esto evita depender solamente de los datos que se ingresan desde la pantalla.

## Rutas

Las rutas permiten navegar entre las páginas de Tetris. Dependiendo de la opción seleccionada, Angular muestra el menú, el perfil, los ajustes, la lista de salas o la pantalla de una partida específica.

Algunas rutas requieren que exista una sesión válida. De esta manera, no se permite entrar a las pantallas del juego como si un jugador estuviera autenticado cuando todavía no ha ingresado.

## Autenticación y sesiones

Proyecto Tetris cuenta con un sistema para reconocer a los jugadores y proteger las operaciones que realizan. La aplicación no utiliza un formulario tradicional de correo y contraseña; permite crear una cuenta con un nombre de usuario único y también ingresar mediante Google.

## Cuentas de jugador

Al registrar una cuenta se utiliza un nickname que no puede estar ocupado por otra persona. El servidor genera una credencial de dispositivo y entrega una sesión que permite realizar solicitudes autorizadas. Esto facilita volver a ingresar sin tener que escribir una contraseña cada vez.

El backend controla que exista una sola sesión activa por usuario. Si se inicia sesión en otro dispositivo, la sesión anterior deja de tener permiso para continuar usando esa cuenta.

## Google y recuperación

El ingreso con Google permite identificar al jugador mediante una cuenta externa. Si se trata de una cuenta nueva, el usuario puede elegir su nickname. También existe la opción de vincular Google con una cuenta que ya se había creado.

Además, el perfil permite utilizar un código de recuperación privado cuando se necesita recuperar el acceso. Estos códigos se verifican en el servidor y no deben incluirse en documentos públicos.

## Guards y permisos

Los guards de Angular revisan si el jugador puede entrar a determinadas rutas. En el backend también se valida el token de sesión y que la persona tenga permiso para realizar la acción que solicita.

Por ejemplo, un invitado no puede modificar las opciones reservadas al creador de la sala, y un jugador no puede mover piezas cuando no le corresponde el turno.

## Módulos del sistema

Proyecto Tetris tiene diferentes módulos que permiten separar las funciones disponibles. Cada modalidad utiliza una parte del mismo motor del juego, pero aplica reglas diferentes según la cantidad de jugadores y la forma en que se utiliza el tablero.

### Solitario

Permite que una persona juegue por su cuenta. Utiliza un tablero de veinte filas por diez columnas, muestra la pieza que está cayendo y calcula puntos por las filas completas y algunas acciones de descenso.

Cada diez filas normales eliminadas aumenta el nivel. La velocidad también cambia conforme avanza la partida. Al terminar, el resultado permite consultar el historial y tratar de superar la mejor puntuación.

### Compartido

Permite jugar de dos a cuatro participantes sobre un mismo tablero. Solo puede mover la pieza quien tenga el turno actual. Cuando una pieza queda colocada, el sistema asigna el turno al siguiente jugador que todavía puede participar.

Cada persona obtiene puntos por sus jugadas, mientras que las filas normales eliminadas aumentan el nivel del tablero común. El creador también puede activar piezas especiales y configurar su probabilidad antes de iniciar.

### Competitivo

Permite jugar de dos a cuatro personas de manera simultánea, cada una sobre su propio tablero. Los jugadores compiten por conseguir la mejor puntuación y no se utilizan piezas especiales.

Cuando alguien pierde, puede observar la partida de los demás. Si queda una sola persona jugando, la ronda no termina automáticamente: continúa hasta que todos hayan perdido. El primer lugar corresponde a la puntuación final más alta y puede compartirse en caso de empate.

### Piezas especiales

En el modo Compartido pueden aparecer piezas con efectos diferentes a los de las piezas normales. El catálogo incluye, entre otros, una bomba que vacía un área de tres por tres, una pieza que elimina una fila y otra que permite obtener el doble de puntos por las filas normales de esa jugada.

También existen efectos que afectan temporalmente a otro jugador, como invertir sus controles de movimiento o aumentar la velocidad de caída. La probabilidad se configura antes de iniciar y los efectos no deben sumar puntos por los bloques retirados artificialmente.

### Pausas y reconexión

En Solitario se puede pausar y reanudar la partida directamente. En las modalidades multijugador se utiliza una votación, donde se requiere más de la mitad de los jugadores habilitados para aprobar la pausa.

Cada jugador dispone de una solicitud de pausa por ronda y la pausa aprobada dura como máximo tres minutos. Si se pierde la conexión, el servidor permite regresar dentro de un plazo de treinta segundos para conservar el estado de la partida.

## Resultados y configuración

Al terminar una ronda, el backend determina las puntuaciones finales y registra la información necesaria para el historial y las estadísticas de cada jugador. También se puede preparar una ronda nueva utilizando la misma sala, cuando corresponde.

Desde los ajustes se pueden modificar preferencias relacionadas con el audio, como activar o desactivar música y efectos, además de ajustar sus volúmenes. Estas opciones permiten adaptar la experiencia del juego a cada persona.

## Publicación y ejecución

Proyecto Tetris puede utilizarse desde un navegador o instalarse como aplicación Android. La página web se publica mediante Firebase Hosting, mientras que el backend continúa ejecutándose en la computadora anfitriona.

Cuando se utiliza el sistema de presentación, el programa comprueba que las herramientas estén disponibles, revisa la conexión con SQL Server, inicia la API y prepara un túnel de Cloudflare. Este túnel proporciona una dirección pública temporal para que los dispositivos se comuniquen con el servidor.

Después, el iniciador verifica la conexión y prepara los archivos web necesarios. Si corresponde publicar una actualización, solicita una confirmación manual con la palabra PUBLICAR. El panel de presentación muestra un enlace y un código QR para abrir el juego desde otros dispositivos.

La aplicación Android utiliza Ionic y Capacitor. Para preparar una versión actualizada se compila Angular, se sincronizan los archivos con Android y se genera una APK firmada. Al actualizar la aplicación debe conservarse el mismo certificado de firma y aumentarse el número interno de versión.

Es importante considerar que Firebase solamente publica los archivos del frontend; no ejecuta SQL Server ni la API. Por eso, para jugar por Internet, la computadora donde está el backend debe permanecer encendida y conectada.

Antes de compartir una actualización, las pruebas del proyecto permiten revisar reglas, salas, autenticación y comunicación. Además, las funciones de audio, controles y conexión deben probarse en dispositivos reales, porque una compilación correcta no garantiza que todos los celulares se comporten exactamente igual.

## Conclusión

El desarrollo de Proyecto Tetris permitió crear un videojuego organizado en diferentes partes, donde cada una cumple una función específica y se comunica con las demás para lograr el funcionamiento general de la aplicación.

El uso de Angular, Ionic, ASP.NET Core, C# y SQL Server permitió desarrollar la parte visual, las reglas del juego y el almacenamiento de información. SignalR hizo posible que varios jugadores reciban actualizaciones durante las partidas y Capacitor permitió preparar una versión para Android.

La división del proyecto por capas facilita comprender dónde se encuentran las clases, servicios y métodos principales. Esto también ayuda cuando se necesita revisar un error, modificar una regla o agregar una mejora sin cambiar todo el programa.

En general, Proyecto Tetris ofrece una modalidad individual y dos modalidades multijugador, con salas, puntuaciones, configuraciones, recuperación de cuentas y resultados. Este manual sirve como referencia para conocer su estructura y continuar dando mantenimiento al sistema.